

100 Important System Design & DSA Interview Questions with Answers

Full-Stack / MERN Developer Interview — concise, practical revision

SYSTEM DESIGN — 30 QUESTIONS

1. What is System Design?

Answer: System design is the process of deciding how components such as clients, servers, APIs, databases, caches and queues work together.

2. What is HLD?

Answer: High-Level Design describes major components, services, data stores and how they communicate.

3. What is LLD?

Answer: Low-Level Design describes detailed modules, classes, functions, interfaces and data structures.

4. What is scalability?

Answer: Scalability is the ability of a system to handle increasing traffic or users while maintaining acceptable performance.

5. Horizontal vs vertical scaling?

Answer: Vertical scaling increases resources of one machine. Horizontal scaling adds more machines and distributes work.

6. What is a load balancer?

Answer: A load balancer distributes incoming requests across multiple servers to improve availability and capacity.

7. What is caching?

Answer: Caching stores frequently used data temporarily so requests can be served faster and database load can be reduced.

8. What is Redis?

Answer: Redis is an in-memory data store commonly used for caching, sessions, counters and queues.

9. What is database indexing?

Answer: An index helps the database find records faster for selected queries, at the cost of storage and write overhead.

10. What is database replication?

Answer: Replication keeps copies of data on multiple database servers to improve availability and read capacity.

11. What is sharding?

Answer: Sharding distributes data across multiple database servers so large datasets and workloads can scale horizontally.

12. What is a CDN?

Answer: A CDN serves static content from locations closer to users, reducing latency.

13. What is an API Gateway?

Answer: An API Gateway is an entry point that can route requests and handle cross-cutting concerns such as authentication and rate limiting.

14. What is rate limiting?

Answer: Rate limiting restricts how many requests a client can make in a period to protect an API from abuse and overload.

15. What is fault tolerance?

Answer: Fault tolerance means a system can continue operating even when some components fail.

16. What is availability?

Answer: Availability is how consistently a system remains accessible and operational.

17. What is latency?

Answer: Latency is the time between sending a request and receiving the response.

18. What is throughput?

Answer: Throughput is the amount of work a system can process in a given time, such as requests per second.

19. Monolith vs microservices?

Answer: A monolith keeps most functionality in one deployable application. Microservices split functionality into independently deployable services.

20. How would you scale a Node.js API?

Answer: Run multiple instances behind a load balancer, optimize queries, use caching, paginate results, move heavy work to background jobs and scale the database.

21. How would you improve a slow API?

Answer: Measure the bottleneck, optimize database queries and indexes, reduce payloads, cache suitable data, paginate large results and remove unnecessary work.

22. How do you design a social networking app?

Answer: Use load balancing, multiple API servers, MongoDB with proper indexes, caching, background workers and real-time messaging where required.

23. How do you design a food-ordering app?

Answer: Separate users, restaurants, menus, carts, orders and payments, then use APIs, database controls, caching and background processing as traffic grows.

24. How would you prevent duplicate orders?

Answer: Use idempotency keys or unique request identifiers so repeated requests cannot create multiple orders.

25. How would you design pagination?

Answer: Use offset pagination for simple cases and cursor-based pagination when datasets are large or frequently changing.

26. How would you handle millions of users?

Answer: Scale horizontally, use load balancers, caching, database replication/sharding where appropriate, efficient queries, queues and monitoring.

27. How would you design authentication?

Answer: Use secure password hashing, authentication tokens or sessions, protected routes, authorization checks and secure transport.

28. How would you handle server failure?

Answer: Use multiple instances, health checks, load balancing, retries where safe, replication and graceful failure handling.

29. What is a message queue?

Answer: A message queue stores tasks or messages so producers and consumers can process work asynchronously.

30. Why use background jobs?

Answer: They move slow tasks such as email, notifications, reports or media processing out of the request-response path.

DSA — 70 QUESTIONS

31. What is Big O notation?

Answer: Big O describes how an algorithm's time or space usage grows as input size increases.

32. What is $O(1)$?

Answer: Constant complexity: the amount of work stays roughly constant as input grows.

33. What is $O(n)$?

Answer: Linear complexity: work grows proportionally with input size.

34. What is $O(\log n)$?

Answer: Logarithmic complexity: the search space is repeatedly reduced, as in binary search.

35. What is $O(n^2)$?

Answer: Quadratic complexity: work often grows from nested loops over the same input.

36. What is an array?

Answer: An array is an ordered collection that provides direct access by index.

37. Find the largest element in an array.

Answer: Scan once while maintaining the maximum. Time $O(n)$, space $O(1)$.

38. Find the second largest element.

Answer: Maintain the largest and second-largest values while scanning once. Time $O(n)$, space $O(1)$.

39. How do you remove duplicates from an array?

Answer: Use a Set for a simple solution. Average time $O(n)$, extra space $O(n)$.

40. How do you find a missing number?

Answer: For numbers $0..n$, compare the expected arithmetic sum with the actual sum, or use XOR. Time $O(n)$, space $O(1)$.

41. What is Two Sum?

Answer: Find two elements whose sum equals a target. A HashMap can solve it in $O(n)$ average time.

42. How do you move zeros to the end?

Answer: Use a pointer to place non-zero values first, then fill remaining positions with zeros. Time $O(n)$, space $O(1)$.

43. What is a palindrome?

Answer: A sequence that reads the same forward and backward.

44. How do you check a palindrome?

Answer: Use two pointers from both ends and compare characters while moving inward.

45. What is an anagram?

Answer: Two strings are anagrams if they contain the same characters with the same frequencies.

46. How do you check an anagram?

Answer: Count character frequencies with a Map or sort both strings and compare.

47. How do you find the first non-repeating character?

Answer: Count frequencies first, then scan again and return the first character with frequency one.

48. What is a HashMap?

Answer: A key-value data structure providing average $O(1)$ lookup, insertion and deletion.

49. What is a Set?

Answer: A collection of unique values that supports efficient membership checks.

50. What is binary search?

Answer: Binary search finds an item in a sorted array by repeatedly halving the search range.

51. What is a stack?

Answer: A LIFO data structure: Last In, First Out.

52. What is a queue?

Answer: A FIFO data structure: First In, First Out.

53. How do you solve Valid Parentheses?

Answer: Push opening brackets onto a stack and match each closing bracket with the top of the stack.

54. What is a linked list?

Answer: A sequence of nodes where each node stores data and a reference to the next node.

55. Array vs linked list?

Answer: Arrays provide fast indexed access. Linked lists provide efficient insertion/deletion when the node position is known, but no direct indexed access.

56. How do you reverse a linked list?

Answer: Use prev, current and next pointers to reverse each link. Time $O(n)$, space $O(1)$.

57. How do you detect a linked-list cycle?

Answer: Use slow and fast pointers. If they meet, a cycle exists.

58. What is a binary tree?

Answer: A tree in which each node has at most two children.

59. What is a BST?

Answer: A Binary Search Tree maintains an ordering between left and right subtree values.

60. What are tree traversals?

Answer: Inorder: Left-Root-Right; Preorder: Root-Left-Right; Postorder: Left-Right-Root; Level-order: breadth-first.

61. BFS vs DFS?

Answer: BFS explores level by level and uses a queue; DFS explores deeply and uses recursion or a stack.

62. What is recursion?

Answer: Recursion is when a function calls itself and must have a base case to stop.

63. What is a base case?

Answer: The base case is the condition that stops recursive calls.

64. What is a graph?

Answer: A graph is a set of vertices connected by edges.

65. Directed vs undirected graph?

Answer: Directed edges have a direction; undirected edges represent a two-way relationship.

66. What is a connected component?

Answer: A connected component is a maximal group of vertices where every vertex is reachable from others within the group.

67. What is a sliding window?

Answer: A technique that maintains a moving range over an array or string to solve many subarray or substring problems efficiently.

68. What is the two-pointer technique?

Answer: It uses two indices that move through a structure to reduce unnecessary comparisons, often achieving $O(n)$ solutions.

69. What is dynamic programming?

Answer: Dynamic programming solves problems with overlapping subproblems by storing results to avoid repeated work.

70. What DSA topics are most important for a junior full-stack interview?

Answer: Focus on arrays, strings, HashMap/Set, two pointers, sliding window, binary search, stack, queue, linked list, trees and Big O.

71. Find the maximum subarray sum.

Answer: Use Kadane's algorithm: maintain the best sum ending at the current position and the overall best sum. Time $O(n)$, space $O(1)$.

72. What is Kadane's algorithm?

Answer: Kadane's algorithm finds the maximum sum of a contiguous subarray in $O(n)$ time.

73. How do you merge two sorted arrays?

Answer: Use two pointers, compare current elements, append the smaller one, then append remaining elements.

74. What is sorting?

Answer: Sorting arranges data according to an ordering such as ascending or descending.

75. Bubble sort complexity?

Answer: Worst and average time are $O(n^2)$; space is $O(1)$.

76. Merge sort complexity?

Answer: Merge sort runs in $O(n \log n)$ time and typically uses $O(n)$ extra space.

77. Quick sort complexity?

Answer: Average time is $O(n \log n)$, worst case $O(n^2)$; extra space depends on implementation and recursion.

78. What is a priority queue?

Answer: A priority queue returns the element with the highest or lowest priority rather than simply the oldest element.

79. What is a heap?

Answer: A heap is a tree-based structure commonly used to implement priority queues. A min-heap keeps the smallest item at the root.

80. What is a trie?

Answer: A trie is a prefix tree used for efficient string and prefix operations.

81. What is memoization?

Answer: Memoization stores results of expensive function calls so repeated inputs can return the cached result.

82. What is dynamic programming's main idea?

Answer: Break a problem into overlapping subproblems, solve each once, and store results for reuse.

83. What is greedy algorithm?

Answer: A greedy algorithm chooses the best immediate option at each step, hoping those choices produce an optimal solution.

84. When is binary search applicable?

Answer: It is applicable when the search space is ordered or has a monotonic property that allows it to be divided reliably.

85. What is a time-space trade-off?

Answer: It means using additional memory to reduce computation time, or saving memory at the cost of more computation.

86. What is recursion stack space?

Answer: Recursive calls consume call-stack memory. Deep recursion can therefore increase space usage or cause stack overflow.

87. How do you find the middle of a linked list?

Answer: Use slow and fast pointers; slow moves one step while fast moves two steps.

88. How do you find the depth of a binary tree?

Answer: Recursively compute 1 plus the maximum depth of the left and right subtrees, or use BFS level counting.

89. What is BFS time complexity?

Answer: For a graph represented with adjacency lists, BFS is $O(V + E)$, where V is vertices and E is edges.

90. What is DFS time complexity?

Answer: For an adjacency-list graph, DFS is $O(V + E)$.

91. What is a priority queue?

Answer: A priority queue removes the highest- or lowest-priority element first; a heap is commonly used to implement it.

92. What is a heap?

Answer: A heap is a tree-based structure where the root follows the heap-order property; it is often used for priority queues.

93. What is a trie?

Answer: A trie is a prefix tree designed for efficient word and prefix lookup.

94. What is memoization?

Answer: Memoization stores previously computed results so repeated inputs can be answered without repeating the calculation.

95. What is a greedy algorithm?

Answer: A greedy algorithm chooses the best immediate option at each step and can be optimal for problems with the right greedy property.

96. What is a time-space trade-off?

Answer: It means using more memory to reduce execution time, or using less memory while doing more computation.

97. How do you find the middle of a linked list?

Answer: Use slow and fast pointers; slow moves one step and fast moves two steps.

98. How do you find the depth of a binary tree?

Answer: Return 1 plus the larger depth of its left and right subtrees, with an empty tree having depth zero.

99. What is the time complexity of BFS?

Answer: With an adjacency-list graph, BFS is $O(V + E)$.

100. What is the time complexity of DFS?

Answer: With an adjacency-list graph, DFS is $O(V + E)$.